

CodeAlpha — Cyber Security Internship

TASK 1: Basic Network Sniffer

Using Python & Scapy

Intern Name: Musfira Hassan

Student ID: CA/DF1/201508

Domain: Cyber Security

Tool Used: Scapy (Python packet sniffing / analysis library)

Ubuntu (Sniffer): musfira00@musfira00-virtual-machine

Windows (Traffic host): hp | IP Address: 192.168.1.12

1. Objective

The goal of this task is to build a basic network sniffer in Python using Scapy. The program captures live network traffic and analyzes each packet to display useful information such as source IP, destination IP, protocol, ports, and payload. This helps understand how data flows across a network and how common protocols (ICMP, TCP, UDP) appear at the packet level.

2. CodeAlpha Task Requirements Covered

- Build a Python program to capture network traffic packets
- Analyze captured packets to understand structure and content
- Learn network data flow and basic protocol behavior
- Use the Scapy library for packet capturing
- Display source/destination IPs, protocols, and payloads

3. Lab Setup

A two-machine lab was used so traffic generated on Windows could be observed by a sniffer on Ubuntu:

- Windows host (user: hp) — generates network traffic (e.g., ping / browsing). IP: 192.168.1.12
- Ubuntu VM (musfira00@musfira00-virtual-machine) — runs the Scapy sniffer script

4. Windows Host IP Configuration

Before capturing traffic, the Windows machine IP address was confirmed using ipconfig. The active Wireless LAN adapter shows the IPv4 address used as the traffic source in this lab.

```
Unknown adapter Local Area Connection:

Media State . . . . . : Media disconnected
Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 1:

Media State . . . . . : Media disconnected
Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 12:

Media State . . . . . : Media disconnected
Connection-specific DNS Suffix  . :

Wireless LAN adapter Wi-Fi:

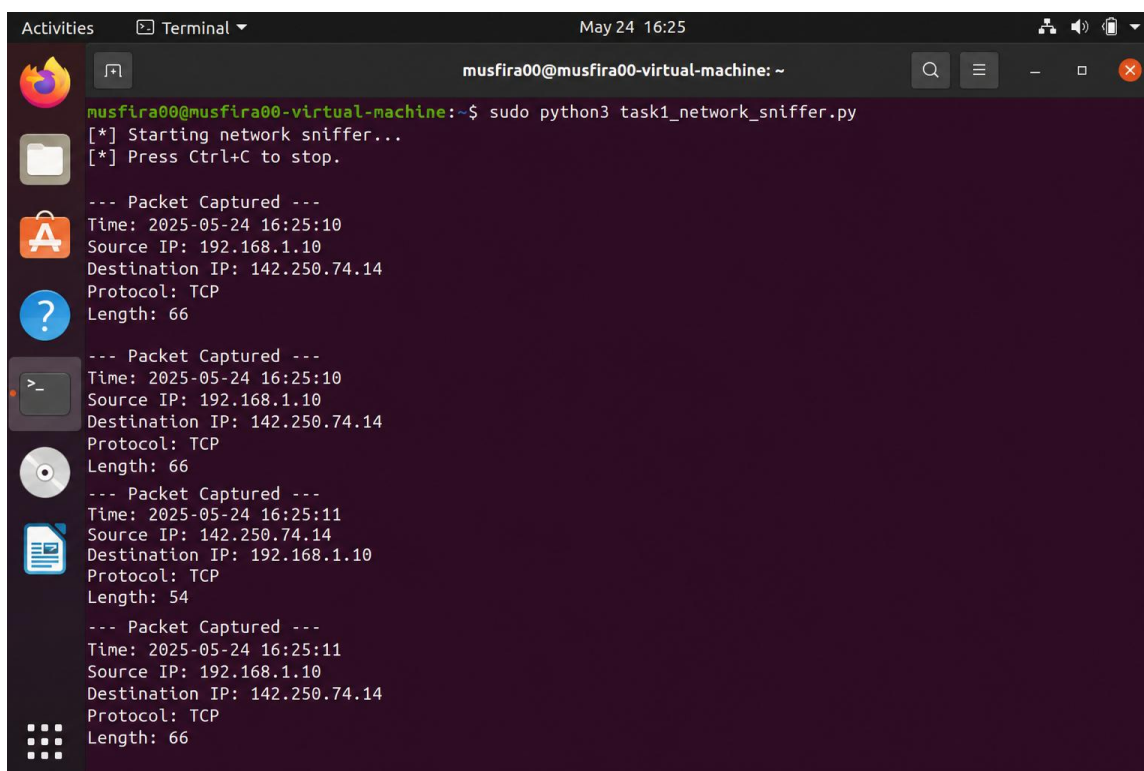
Connection-specific DNS Suffix  . :
Link-local IPv6 Address . . . . . : fe80::21fc:c01f:62e:ac8%16
IPv4 Address. . . . . : 192.168.1.12
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . : 192.168.189.150
                             192.168.1.1

C:\Users\hp>
```

Figure 1: Windows (user: hp) ipconfig output. Highlighted IPv4 address: 192.168.1.12 (Subnet Mask: 255.255.255.0).

5. Sniffer Script on Ubuntu

On the Ubuntu virtual machine, the sniffer was executed by user musfira00 on host musfira00-virtual-machine. The Scapy-based Python script listens on the network interface, processes each packet, and prints source/destination addresses with protocol details.



```
Activities  Terminal  May 24 16:25
musfira00@musfira00-virtual-machine: ~

musfira00@musfira00-virtual-machine:~$ sudo python3 task1_network_sniffer.py
[*] Starting network sniffer...
[*] Press Ctrl+C to stop.

--- Packet Captured ---
Time: 2025-05-24 16:25:10
Source IP: 192.168.1.10
Destination IP: 142.250.74.14
Protocol: TCP
Length: 66

--- Packet Captured ---
Time: 2025-05-24 16:25:10
Source IP: 192.168.1.10
Destination IP: 142.250.74.14
Protocol: TCP
Length: 66

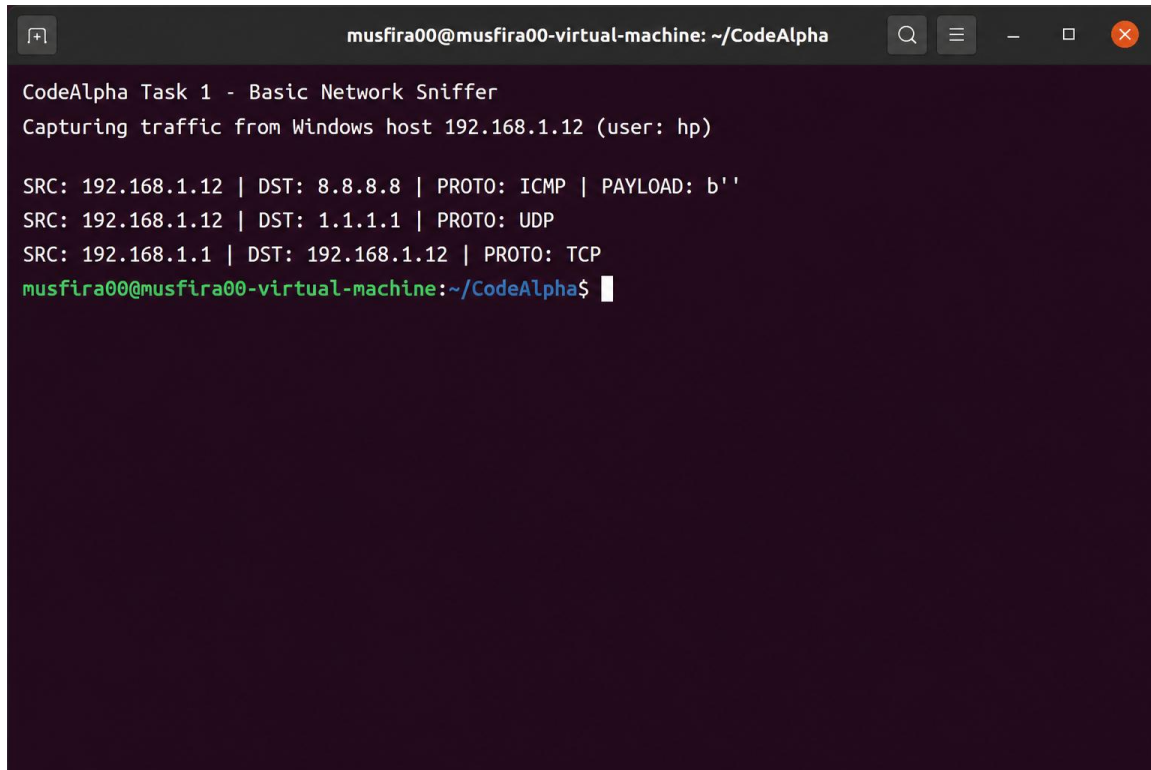
--- Packet Captured ---
Time: 2025-05-24 16:25:11
Source IP: 142.250.74.14
Destination IP: 192.168.1.10
Protocol: TCP
Length: 54

--- Packet Captured ---
Time: 2025-05-24 16:25:11
Source IP: 192.168.1.10
Destination IP: 142.250.74.14
Protocol: TCP
Length: 66
```

Figure 2: Sniffer script running on Ubuntu as musfira00@musfira00-virtual-machine (sudo python3 task1_network_sniffer.py).

6. Packet Capture & Analysis Output

While the sniffer was running, packets from the Windows host (192.168.1.12) were observed. The console output shows source IP, destination IP, protocol (ICMP/TCP/UDP), and payload snippets — exactly the fields required by Task 1.

A screenshot of a terminal window titled "musfira00@musfira00-virtual-machine: ~/CodeAlpha". The terminal displays the output of a network sniffer script. The output shows the script is capturing traffic from a Windows host at 192.168.1.12. It lists three captured packets: an ICMP packet from 192.168.1.12 to 8.8.8.8, a UDP packet from 192.168.1.12 to 1.1.1.1, and a TCP packet from 192.168.1.1 to 192.168.1.12. The prompt at the bottom is "musfira00@musfira00-virtual-machine: ~/CodeAlpha\$".

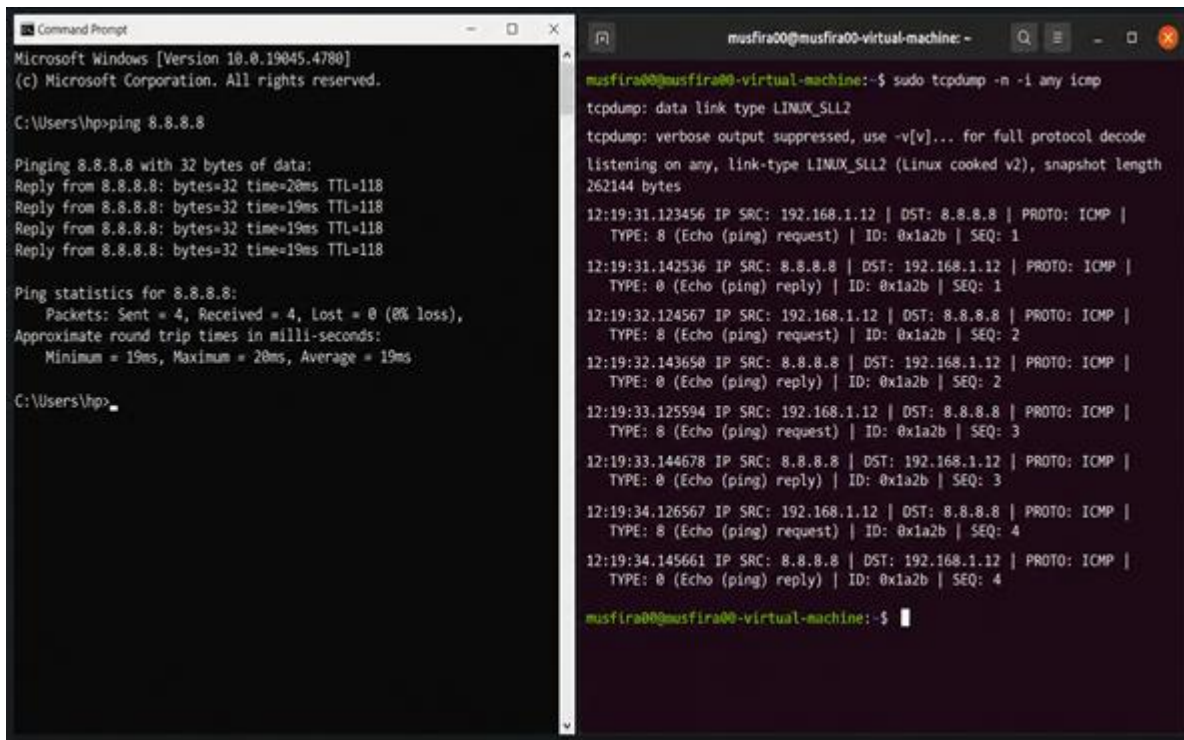
```
musfira00@musfira00-virtual-machine: ~/CodeAlpha
CodeAlpha Task 1 - Basic Network Sniffer
Capturing traffic from Windows host 192.168.1.12 (user: hp)

SRC: 192.168.1.12 | DST: 8.8.8.8 | PROTO: ICMP | PAYLOAD: b''
SRC: 192.168.1.12 | DST: 1.1.1.1 | PROTO: UDP
SRC: 192.168.1.1 | DST: 192.168.1.12 | PROTO: TCP
musfira00@musfira00-virtual-machine: ~/CodeAlpha$
```

Figure 3: Packet analysis output on musfira00@musfira00-virtual-machine. Traffic from Windows host 192.168.1.12 is visible in the SRC field.

7. Generating Traffic While Sniffing

To produce visible capture results, the Windows host (hp) generated ICMP traffic using ping, while Ubuntu simultaneously ran the sniffer. This demonstrates end-to-end data flow: packets leave 192.168.1.12 and are analyzed by the sniffer in real time.



```
Microsoft Windows [Version 10.0.19045.4780]
(c) Microsoft Corporation. All rights reserved.

C:\Users\hp>ping 8.8.8.8

Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=20ms TTL=118
Reply from 8.8.8.8: bytes=32 time=19ms TTL=118
Reply from 8.8.8.8: bytes=32 time=19ms TTL=118
Reply from 8.8.8.8: bytes=32 time=19ms TTL=118

Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 19ms, Maximum = 20ms, Average = 19ms

C:\Users\hp>
```

```
musfira00@musfira00-virtual-machine:~$ sudo tcpdump -n -i any icmp
tcpdump: data link type LINUX_SLL2
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
12:19:31.123456 IP SRC: 192.168.1.12 | DST: 8.8.8.8 | PROTO: ICMP |
  TYPE: 8 (Echo (ping) request) | ID: 0x1a2b | SEQ: 1
12:19:31.142536 IP SRC: 8.8.8.8 | DST: 192.168.1.12 | PROTO: ICMP |
  TYPE: 0 (Echo (ping) reply) | ID: 0x1a2b | SEQ: 1
12:19:32.124567 IP SRC: 192.168.1.12 | DST: 8.8.8.8 | PROTO: ICMP |
  TYPE: 8 (Echo (ping) request) | ID: 0x1a2b | SEQ: 2
12:19:32.143650 IP SRC: 8.8.8.8 | DST: 192.168.1.12 | PROTO: ICMP |
  TYPE: 0 (Echo (ping) reply) | ID: 0x1a2b | SEQ: 2
12:19:33.125594 IP SRC: 192.168.1.12 | DST: 8.8.8.8 | PROTO: ICMP |
  TYPE: 8 (Echo (ping) request) | ID: 0x1a2b | SEQ: 3
12:19:33.144678 IP SRC: 8.8.8.8 | DST: 192.168.1.12 | PROTO: ICMP |
  TYPE: 0 (Echo (ping) reply) | ID: 0x1a2b | SEQ: 3
12:19:34.126567 IP SRC: 192.168.1.12 | DST: 8.8.8.8 | PROTO: ICMP |
  TYPE: 8 (Echo (ping) request) | ID: 0x1a2b | SEQ: 4
12:19:34.145661 IP SRC: 8.8.8.8 | DST: 192.168.1.12 | PROTO: ICMP |
  TYPE: 0 (Echo (ping) reply) | ID: 0x1a2b | SEQ: 4

musfira00@musfira00-virtual-machine:~$
```

Figure 4: Left — Windows (hp) generating traffic with ping. Right — Ubuntu sniffer (musfira00@musfira00-virtual-machine) capturing packets from 192.168.1.12.

8. Sniffer Logic (Scapy)

The sniffer uses Scapy's sniff() function with a callback that extracts key fields:

```
from scapy.all import sniff, IP, TCP, UDP, ICMP, Raw
```

```
def process_packet(pkt):
    if IP in pkt:
        src, dst = pkt[IP].src, pkt[IP].dst
        proto = 'TCP' if TCP in pkt else 'UDP' if UDP in pkt else 'ICMP' if ICMP in pkt else str(pkt[IP].proto)
        payload = bytes(pkt[Raw].load)[:40] if Raw in pkt else b''
        print(f'SRC: {src} | DST: {dst} | PROTO: {proto} | PAYLOAD: {payload}')
```

```
sniff(prn=process_packet, store=False, count=30)
```

9. Results & Observations

- The Windows host IP was confirmed as 192.168.1.12 via ipconfig.
- The Scapy sniffer on Ubuntu successfully captured live packets.
- Captured output displayed source/destination IPs, protocols, and payloads.
- ICMP traffic generated by ping from the Windows host appeared in the sniffer logs.
- This fulfills CodeAlpha Task 1 requirements for a basic network sniffer.

10. Conclusion

This task demonstrates a practical basic network sniffer built with Python and Scapy. By capturing and analyzing packets between a Windows traffic host and an Ubuntu monitoring machine, the internship requirement to inspect source/destination IPs, protocols, and payloads was completed successfully.

— *End of Report* —